ROS2 Humble Hawksbill Report

Samed Chodankar

***Abstract*—This report presents a detailed study of Robot Operating System 2 (ROS2) Humble and its application in robotic software development. The work was completed as part of the Driverless Taskphase, where Ubuntu 22.04 was installed and ROS2 Humble was configured. The report explains the ROS2 computation graph, communication concepts including nodes, topics, services, actions, and parameters, Quality of Service settings, custom interfaces, colcon build system, and workspace organization. Two practical implementations are described. The first solution demonstrates custom message communication for calculating wheel longitudinal velocity using angular velocity and radius. The second solution implements a three-node communication architecture for palindrome checking using custom string messages.**

I. Introduction

ROS2 is an open-source framework for building robotic applications. A robot is rarely a single program. It usually has sensors, controllers, a navigation stack and actuators, and all of these have to pass data to one another reliably. ROS2 handles that by letting each part run as its own small program, called a node, and giving those nodes standard ways to talk.

For this taskphase I used the ROS2 Humble distribution. I first set up Ubuntu, then installed ROS2 from the Debian packages and configured the environment so that every new terminal could find the ROS2 tools. After that I worked through the beginner tutorials, which cover the command line tools, turtlesim, rqt, nodes, topics, services, parameters, actions, logging, launch files, packages and custom interfaces. Most of what follows comes from those tutorials and from the two solutions I wrote at the end.

The rest of the paper goes through the main ideas one at a time, starting with the computation graph and the communication types, and then covers Quality of Service, the build system and custom interfaces. The two practical solutions are described after that.

II. Installation and Environment Setup

The first step was getting a working Ubuntu 22.04 installation, since Humble targets that release. After that I followed the official installation guide for the Debian packages. This means enabling the required software repositories, adding the ROS2 package key and apt source, updating the package index and installing the desktop version of Humble, which includes the visualization tools and the tutorial packages.

Installing the packages is not enough on its own, because the shell has to be told where ROS2 lives. This is done by sourcing the setup file that comes with the installation. Doing it by hand in every terminal gets tiresome, so the usual approach is to add the command to the shell startup file. To check that everything worked, I ran the demo talker and listener nodes in two terminals and confirmed that the listener printed the messages sent by the talker.

III. Command Line Tools and Turtlesim

A large part of the tutorials is about the ros2 command line tool, which can inspect almost every part of a running system. It can list the active nodes and topics, print the messages on a topic, call a service, read and set parameters and send action goals. Because of this, a system can be explored and debugged from the terminal without writing any extra code.

The tutorials use turtlesim, a small simulator in which a turtle moves around a window, to make these ideas easier to see. The turtle is driven by messages on a topic, it has services for things like clearing the screen, and it has parameters such as the background colour. The rqt tools sit on top of this and give a graphical way to look at the same information, including the graph of nodes and topics.

IV. ROS2 Computation Graph

The computation graph is the network of everything that is running in a ROS2 system and the links between those things. Every running program shows up as a node, and the links are the topics, services and actions the nodes use to exchange information. It is a useful way to see how the processes fit together without reading any source code.

When something does not work, the graph is usually the first place to look. A subscriber listening on a misspelled topic name, for example, simply shows up as a node with no connection. Tools such as rqt_graph draw this picture automatically, and the ros2 command line tools can list the nodes and topics that are currently active. On a large robot with dozens of nodes running at the same time, being able to see the whole structure makes both debugging and design much easier.

V. ROS2 Nodes

A node is the basic unit of computation in ROS2, and each one is supposed to do a single job. A camera node might do nothing except publish images, while another node takes those images and looks for objects in them. Neither of them needs to know how the other is written.

Splitting a system into many small nodes instead of one big program has real advantages. Each node can be started, stopped and tested by itself, and one that crashes does not have to bring the others down with it. A well written node can also be reused in a completely different robot as long as the data it expects stays the same. The cost is that the nodes have to agree on how they communicate, which is where topics, services and actions come in.

VI. Topics

Topics are channels for data that keeps flowing. A publisher sends messages to a named topic, and any number of subscribers can receive them. The publisher does not know who is listening, and in fact does not care whether anybody is.

This communication is asynchronous, so the publisher sends its data and moves on without waiting for an answer. That suits sensor readings, robot state and control commands, where fresh data keeps arriving and the old data quickly stops mattering. Typical examples are wheel speed, camera images, laser scanner readings and the position of the robot.

VII. Services

A service is a request and response pair. One node, the client, sends a request to another node, the server, which does the work and sends a reply back. The client normally waits for that reply before carrying on.

This is the better choice when something only happens now and then and the caller needs to know the result. Resetting a sensor, changing a setting on the robot or asking for one specific calculation are all good examples. A service is not meant for streaming data, because every call is a separate request.

VIII. Topics vs Services

The main difference between the two is how they communicate. Topics follow a publisher-subscriber pattern and fit continuous streams, while services follow a client-server pattern and fit requests that need an immediate answer. A topic is like a radio broadcast that anyone can tune into, whereas a service is more like a phone call with a question and an answer.

A camera is a good case for a topic, since it produces images all the time and several nodes may want them. Asking the robot to compute a particular value is better as a service because the request only comes up when somebody needs the answer. Choosing the wrong one usually works at first but leads to wasted traffic or missed data later.

IX. Actions and Parameters

Actions are meant for tasks that take a long time, where the caller wants updates while the work goes on and may want to cancel it. An action has three parts: a goal, some feedback while the task runs, and a final result. Navigation is the usual example. A robot is told to drive to a location, reports its progress on the way, and finally says whether it got there.

Parameters are values stored inside a node that can be changed without touching the source code. Speed limits, sensor thresholds and controller gains are typical examples. Being able to change these when the node is launched, or even while it is running, saves a lot of rebuilding when values need tuning.

X. Quality of Service Settings(QoS)

ROS2 adds Quality of Service (QoS) settings, which let the developer control how messages are stored, delivered and handled between a publisher and a subscriber. This was one of the bigger changes compared with the first version of ROS.

The main policies are reliability, durability, history and depth. Reliable communication makes sure messages are delivered and will resend them if needed, while best-effort communication just sends the data as quickly as possible and accepts that some may be lost. Durability decides whether a subscriber that joins late can still receive messages that were sent earlier. History and depth decide how many messages are kept in the queue.

These settings matter because robots work in very different conditions. A camera stream is better off with fast delivery, since a lost frame is replaced by the next one almost immediately. Control commands, on the other hand, should arrive every time. A publisher and a subscriber also need compatible QoS settings, or they will not connect at all.

XI. Colcon and Workspace Structure

Colcon is the standard build tool for ROS2. It compiles the packages, works out the order in which they depend on one another and produces the executables that are run later.

A ROS2 workspace has a few folders that colcon creates and uses. The src folder holds the source code of the packages. The build folder holds temporary files from compilation. The install folder holds the finished executables and the setup files that have to be sourced before anything can be run. The log folder keeps information about earlier builds, which helps when a build fails.

Keeping the workspace organized makes it practical to work on several packages at once. It also means a single command can rebuild everything after a change. The install setup file has to be sourced after a build, because otherwise the new packages cannot be found.

XII. Custom Interfaces

The standard message types do not cover everything. When they are not enough, ROS2 lets the developer define custom messages and services that describe exactly what data is exchanged between nodes. These definitions are plain text files, and the build system generates the code from them.

I created a custom message for each of the two solutions. The TaskOne message stores an angular velocity and a wheel radius, and RT2String stores one string entered by the user.

XIII. Logging and Launch Files

ROS2 has a built-in logging system, and nodes report messages at different severity levels such as debug, info, warning, error and fatal. The level can be chosen so that only the important messages are shown while a robot is running, and the detailed ones can be switched on when something needs to be investigated. This is much cleaner than printing text from the code and hoping it is not lost in the noise.

Launch files solve a different problem. A real robot may need many nodes running together, each with its own parameters and settings, and starting them one by one in separate terminals would be slow and error-prone. A launch file describes the whole set in one place, so that a single command starts everything. This becomes more important as a project grows.

XIV. Solution 1: TaskOne Message Implementation

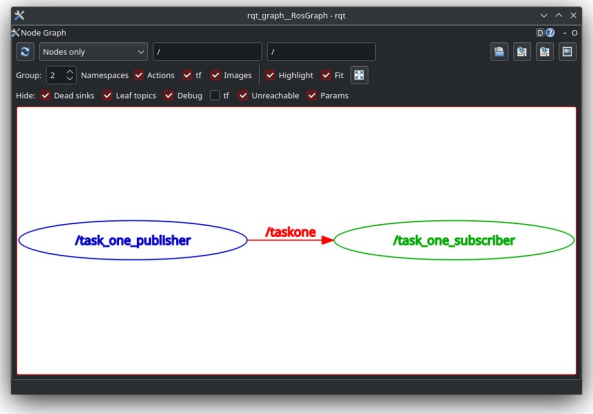
The first problem asked for a custom message called TaskOne with two floating point fields, angVel and radius.

The publisher node asks for the angular velocity and the wheel radius in the terminal. It puts the two numbers into a TaskOne message and publishes it on the /taskone topic.

The subscriber node listens on /taskone. Each time a message arrives it calculates the longitudinal speed of the wheel using the equation

Longitudinal Speed = Angular Velocity × Radius

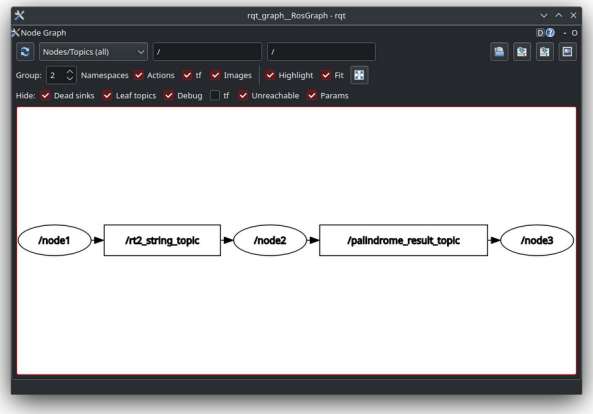
and prints the result in the terminal. The ROS graph for this solution is simple: the publisher node is connected to the subscriber node through the /taskone topic, and nothing else is involved.



XV. Solution 2: Three Node Communication System

The second solution uses three independent nodes that pass data along a chain.

Node 1 keeps asking the user for a string and publishes each one using the custom RT2String message, which contains a single string field.



Node 2 subscribes to that topic and checks whether the string it receives is a palindrome. If it is, Node 2 publishes the value 1, and if not it publishes 0.

Node 3 receives that result from Node 2. When the value is 1 it responds with Yes, and when the value is 0 it responds with No. The answer is printed in the terminal.

The ROS graph is therefore a straight line: the string publisher feeds the string topic, which feeds the palindrome checker, which publishes on the result topic, which finally feeds the response node. Splitting the work this way means each node has only one small job, and any of them could be replaced without changing the others.

XVI. Conclusion

This task gave me practical experience with how ROS2 is structured and how its parts communicate. Writing custom messages, publishers, subscribers and a multi-node system showed how a robotic application can be built from small and independent pieces.

Understanding computation graphs, QoS settings, interfaces and workspace management is important for building reliable autonomous systems, and the two solutions show how flexible and scalable ROS2 is even for small tasks. The same ideas should carry over to larger projects, where many more nodes have to work together.

References

1. https://docs.ros.org/en/humble/
2. https://colcon.readthedocs.io/